

Capítulo 9 Aprendizaje No Supervisado

El capítulo se basa en:

- agrupar instancias similares (clustering)
- detectar lo que se sale de lo normal (detección de anomalías)
- estimar cómo se distribuyen los datos (estimación de densidad).

En clustering no existen categorías definidas antes. El algoritmo tiene que descubrir los grupos por sí solo, y no hay una única forma correcta de definir qué es un grupo: algunos métodos buscan puntos cercanos a un centro, otros buscan zonas densas, otros construyen jerarquías. Sus usos son variados: segmentar clientes, explorar datos, reducir variables o detectar anomalías.

K-Means es el método más simple y el más usado. Hay que indicarle cuántos grupos se buscan. El proceso es sencillo: coloca centros al azar, asigna cada punto al centro más cercano, recalcula cada centro como el promedio de sus puntos y repite hasta que ya no cambian. Siempre llega a un resultado, pero no siempre al mejor posible, por lo que conviene repetir el proceso varias veces y quedarse con la mejor solución.

El indicador llamado inercia mide qué tan lejos están los puntos de su centro, pero baja de forma automática mientras más grupos se agreguen, de modo que no sirve por sí solo. Una forma práctica de decidir es observar en qué punto deja de bajar con rapidez (el método del codo). Otra más precisa aunque más costosa de calcular es el coeficiente de silueta que compara qué tan bien encaja cada punto en su propio grupo frente a qué tan cerca está del grupo vecino.

K-Means tiene límites claros, funciona mal cuando los grupos tienen formas, tamaños o densidades muy distintas, esto porque asume que todos son más o menos redondos y con tamaño parecido. Es sensible a la escala de las variables, por lo que conviene normalizarlas antes de aplicarlo.

Usos prácticos:

- simplificar los colores de una imagen agrupando píxeles parecidos.
- usar clustering como paso previo a un modelo supervisado (convertir cada dato en su distancia a distintos grupos puede mejorar el desempeño de un clasificador)
- cuando hay pocos datos etiquetados se agrupan todos los datos, se etiqueta a mano solo el ejemplo más representativo de cada grupo, y esa etiqueta se extiende al resto del grupo.

DBSCAN parte de una idea distinta: define un grupo como una zona continua de alta densidad, sin depender de un centro. Su ventaja principal es que puede detectar grupos de forma irregular y no necesita que se indique cuántos grupos existen. A cambio, depende bastante de qué tan cerca deben estar los puntos para considerarse vecinos y no tiene una forma directa de clasificar datos nuevos.

Los Modelos de Mezcla Gaussiana asumen que los datos provienen de varias distribuciones normales superpuestas, cada una representando un grupo con su propia forma y orientación. El entrenamiento alterna entre estimar a qué grupo pertenece probablemente cada punto y ajustar los parámetros de cada distribución según esas estimaciones. Además de agrupar, este modelo permite generar datos nuevos y detectar anomalías, identificando los puntos que caen en zonas de muy baja probabilidad.

Conclusion, el aprendizaje no supervisado no es un ejercicio aislado, se combina constantemente con el aprendizaje supervisado (ya sea creando variables nuevas, aprovechando datos sin etiquetar o vigilando sistemas en busca de comportamientos anómalos). Comparar K-Means, DBSCAN y los Modelos de Mezcla Gaussiana también deja clara una lección práctica: la definición de "grupo" cambia según el método, y elegir el correcto depende de la forma real que tengan los datos de cada proyecto.

Capítulo 10 Redes Neuronales con Keras

Este capítulo abre la segunda parte del libro, y es dedicada al Deep Learning: cómo construir redes neuronales artificiales y entrenarlas con Keras.

Un poco de historia

- Las redes neuronales surgieron en los años 40, tuvieron un primer auge y luego cayeron en desinterés por décadas, tras demostrarse que el modelo original no podía resolver problemas simples pero no lineales.
- Resurgieron gracias a más datos disponibles, más poder de cómputo y mejores algoritmos de entrenamiento.

Del Perceptrón al MLP

- El Perceptrón original es una sola capa que decide "sí" o "no" según una suma ponderada de entradas. Funciona como un clasificador lineal simple.
- Su limitación principal: no puede resolver problemas que no se separan con una línea recta.
- La solución fue apilar varios perceptrones en capas: así nace el MLP (Perceptrón Multicapa), con una capa de entrada, capas ocultas y una capa de salida.
- Cuando hay muchas capas ocultas apiladas, se le llama red profunda — de ahí el nombre Deep Learning.

Backpropagation: lo que hizo posible entrenar estas redes

- Es Descenso de Gradiente visto antes, aplicado capa por capa mediante dos pasadas: una hacia adelante (calcula la predicción) y otra hacia atrás (reparte el error entre todas las conexiones).
- Para que funcione se necesitan funciones de activación suaves en vez de un "sí/no". Las más usadas son sigmoide, tanh y mayoritariamente ReLU.

- Sin esa no linealidad entre capas, por más capas que se apilen, la red seguiría comportándose como un modelo lineal simple.

Cómo se adapta la salida según la tarea

- Regresión: una neurona de salida por valor a predecir, normalmente sin restricciones.
- Clasificación binaria: una neurona con salida tipo probabilidad.
- Clasificación multiclase: una neurona por categoría, con probabilidades que suman 1 entre todas.

En la práctica con Keras

- Se arma la red apilando capas: aplanar la entrada, capas densas con ReLU, y una capa final de salida.
- El número de parámetros crece muy rápido / una sola capa mediana ya puede sumar cientos de miles.
- Se define la función de error, el optimizador, y las métricas a monitorear; se entrena vigilando un conjunto de validación aparte para detectar sobreajuste a tiempo.

Ajustar el tamaño de la red ("un arte oscuro" según el escritor)

- Las redes profundas son más eficientes que una sola capa ancha: aprovechan que los datos tienen estructura por niveles, algo simple primero, algo más complejo después.
- Ya no se recomienda una forma "piramidal" de capas; usar el mismo tamaño en todas suele funcionar igual o mejor.
- Estrategia práctica común: usar una red más grande de lo necesario y luego controlar el sobreajuste con otras técnicas, en vez de buscar el tamaño "perfecto" desde el inicio.

Conclusión, la transición entre el ML más clásico e interpretable y el Deep Learning que gana su poder apilando muchas capas simples entrenadas con la misma idea de siempre: bajar el error paso a paso.